# Acknowledgement

We would like to express our sincere gratitude to our supervisor, **Mr. Nguyen The Hoang**, for his continuous guidance, technical feedback, and patience throughout the development of THE LAST SIGNAL.

We would also like to extend our heartfelt thanks to **FPT University Ho Chi Minh City**, our lecturers, our families, our friends, and our teammates for their encouragement, support, and valuable contributions throughout this journey. Their motivation has been an important source of inspiration, enabling us to remain committed and to successfully complete this capstone project.

# Definition and Acronyms

The table below lists the acronyms and domain-specific terms used throughout this document.

| Acronym / Term | Definition |
|---|---|
| NGO | Unity Netcode for GameObjects: the multiplayer networking framework used |
| UTP | Unity Transport Package |
| Expedition | One match aboard the derelict vessel: three survivors against The Shade |
| Survivor | One of the three players who repair the vessel and evacuate (Drone Operator, Lead Engineer, Mechanic) |
| The Shade | The hostile entity, controlled by a player in ranked matches or by a finite state machine in practice mode |
| Dedicated Server | A headless build of the game that runs a match without a local player and is authoritative over its state |
| Orchestrator | Backend service that starts, monitors, and stops game server instances |
| Join Token | A single-use, expiring token that lets one player connect to one game server instance |
| Hidden Information | Game state that a player must not receive because their side is not allowed to see it |
| Matchmaking | Grouping queued players into balanced matches |
| Rating / Glicko-2 | A number that estimates player skill; Glicko-2 also tracks the uncertainty and volatility of that estimate |
| PvP / PvE | Player versus player / player versus environment |
| Light Burn | Damage that strong light inflicts on The Shade; at a threshold it stuns The Shade and forces a retreat |
| Admin Portal | Web application that administrators use to manage players, servers, and balance settings |
| FSM | Finite State Machine |
| FOV | Field of View: raycast-based vision detection |
| NavMesh | Navigation Mesh used for AI pathfinding |
| URP | Universal Render Pipeline |
| RPC | Remote Procedure Call |
| HUD | Heads-Up Display |
| API / REST | Application Programming Interface / Representational State Transfer, a common style for web APIs |
| JWT | JSON Web Token, used for signed-in player sessions |
| CI/CD | Continuous Integration / Continuous Delivery |

| Acronym / Term | Definition |
|---|---|
| FR / NFR | Functional Requirement / Non-Functional Requirement |
| MVP | Minimum Viable Product |
| SRS / SAD / GDD | Software Requirement Specification / Software Architecture Document / Game Design Document |
| UC | Use Case |
| GUI | Graphical User Interface |

# I. Project Introduction

THE LAST SIGNAL is a 2D top-down asymmetric multiplayer horror game developed as an FPT University capstone project, together with the online platform that runs it. In each match, three survivors repair a derelict space vessel and evacuate while a fourth player, The Shade, hunts them in the dark. Matches run on dedicated game servers that a backend platform starts on demand. The same platform manages accounts, matchmaking, ratings, and administration. This section introduces the project team, the market context, the product vision, and the scope.

## 1. Overview

### 1.1 Project Information

- **Project name:** THE LAST SIGNAL: A 2D Top-Down Asymmetric Multiplayer Horror Game with Dedicated Game Servers, Ranked Matchmaking, and an Online Service Platform.
- **Capstone project code:** FA26SE / GFA26SE54.
- **Group name:** GFA26SE54.
- **Duration:** 09/2026 to 03/2027 (approximately 15 working weeks).
- **Software type:** Unity game client (Windows), dedicated game server (Linux, headless Unity build), backend web service with database, and web-based admin portal.

### 1.2 Project Team

| Full Name | Role | Email | Mobile |
|---|---|---|---|
| Nguyễn Thế Hoàng | Supervisor | hoangnt20@fe.edu.vn | 0986 628 525 |
| Cao Thành Danh | Leader | danhcao270305@gmail.com | 0983 713 415 |
| Nguyễn Công Thiên Ân | Member | nguyencongthienan1@gmail.com | 0879 294 494 |
| Hồ Trọng | Member | ht147652@gmail.com | 0397 523 470 |
| Phan Doãn Thuận | Member | thuanphan147159@gmail.com | 0387 371 761 |

## 2. Product Background

Survival-horror games built around darkness, scarce resources, and asymmetric roles have become one of the fastest-growing indie genres in recent years, driven by titles such as Lethal Company and Phasmophobia. These games succeed by forcing small groups of players to communicate under pressure, turning simple mechanics (a flashlight, a locked door, a shared resource) into sources of genuine tension.

However, most existing titles give every player an identical toolkit and let them freely choose what to do moment to moment. Few games assign each player a distinct, mechanically enforced role where the tools one player carries only matter in combination with what another player is doing at the same time.

THE LAST SIGNAL explores this gap aboard a derelict space mining vessel. Three survivors work as a team. One operates a recon drone that provides the ship's only reliable source of light, while the Lead Engineer and the Mechanic repair systems together, in paired repairs that require both of them to act at the same time. The fourth player is The Shade, a hostile entity that hunts the survivors in the dark and is weakened by strong light. The survivors win by reactivating the reactor and evacuating through the Airlock before a countdown ends. The Shade wins by stopping them.

Putting a human on the hostile side adds a second requirement that small cooperative games can avoid. When one player hunts three others, fairness matters. The player who runs the match must not be able to cheat, information that a side should not have (such as where The Shade is) must stay hidden, and results must be trustworthy enough to rank players. Small cooperative games usually let one player's machine act

as the server. Competitive games use dedicated servers and a service backend for accounts, matchmaking, and ratings.

THE LAST SIGNAL is therefore delivered as a ranked asymmetric game on dedicated servers, with the online platform as a necessary part of the product rather than decoration. The same match engine also provides a practice mode against an AI-controlled Shade, so the game remains playable when few players are online.

## 3. Existing Systems

### 3.1 Dead by Daylight (Behaviour Interactive, 2016)

An asymmetric survival horror game in which one Killer hunts four Survivors. It is the best-known 4-versus-1 title and the closest market model for THE LAST SIGNAL.

- **Strengths:** It runs on dedicated servers and uses a hidden skill-based matchmaking rating for ranked matches. The rating is based on a customized Glicko-2 system, with one shared value for all Survivors and a separate value for each Killer. Progression through in-game currency keeps players returning.
- **Limitations:** Balance between the two sides is a constant point of debate. The player base is split across separate queues, so wait times and match fairness vary. Survivors have no fixed roles, so cooperation is not structurally enforced.

### 3.2 Evolve (Turtle Rock Studios, 2015)

Another 4-versus-1 game, in which four hunters track one player-controlled monster. It is mainly a cautionary example: sources on asymmetric multiplayer note that such games struggle to keep a community and to balance the two sides. THE LAST SIGNAL handles this risk by building the ranked mode on a match engine that also works against an AI-controlled Shade.

### 3.3 Lethal Company (Zeekerss, 2023)

An indie cooperative survival horror game where players explore abandoned moons to scavenge scrap while avoiding lethal hazards and monsters.

- **Strengths:** High emergent tension, strong risk/reward decision-making, effective atmosphere using lighting and darkness, and compelling voice chat dynamics.
- **Limitations:** Players share largely homogeneous skill sets without rigid role enforcement, and the monsters are controlled by the game, not by a player.

### 3.4 Phasmophobia (Kinetic Games, 2020)

A 4-player online co-op psychological horror game where players act as paranormal investigators identifying ghost types.

- **Strengths:** Excellent sanity and investigation dynamics, high replayability, and immersive audio-visual tension.
- **Limitations:** Passive gameplay loop for some players and limited mechanical coordination required during active survival phases.

### 3.5 Overcooked (Ghost Town Games, 2016)

A chaotic cooperative cooking game where players work together under time limits to prepare and deliver meals.

- **Strengths:** Strict role division, interdependent task chains, and high demand for active communication and coordination under pressure.
- **Limitations:** Lacks horror elements, atmospheric immersion, or an opposing player, operating purely as an arcade time-management puzzle.

### 3.6 Player-hosted and dedicated-server models: Valheim, Palworld

Host-based co-op is the common model for smaller cooperative games because it is cheaper to build and needs no backend. Dedicated servers appear when a game needs more than four players, a world that stays online when the host leaves, or administrative control. Palworld, for example, supports four players in invite co-op and up to 32 on a dedicated server, and Valheim offers both in-game hosting and dedicated servers. Competitive games add another reason: a host who controls the match can cheat, so the results cannot be trusted. THE LAST SIGNAL uses dedicated servers for ranked matches for this reason.

### 3.7 Market positioning

The figure places the reference games and THE LAST SIGNAL on two axes. It is an illustrative positioning based on the team's judgment, not measured data.

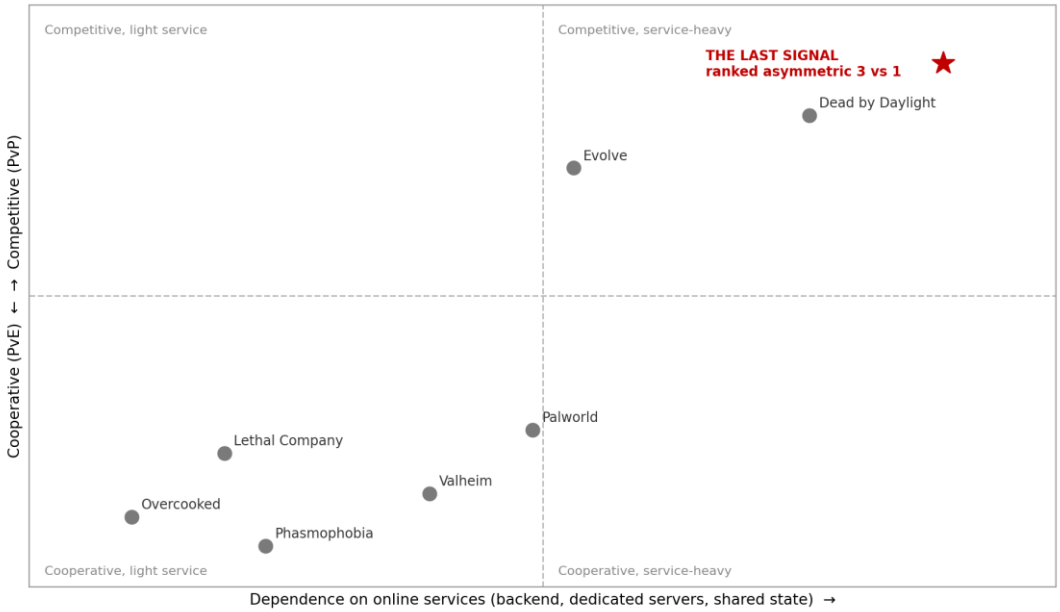



*Figure 1. Illustrative positioning of reference games and THE LAST SIGNAL.*

| Game | Studio, year | What it illustrates |
|---|---|---|
| Dead by Daylight | Behaviour Interactive, 2016 | Ranked asymmetric multiplayer on dedicated servers with a hidden skill rating |
| Evolve | Turtle Rock Studios, 2015 | Risk: balance and community in 4-versus-1 games |
| Lethal Company | Zeekerss, 2023 | Atmosphere, light and darkness; host-based co-op baseline |
| Phasmophobia | Kinetic Games, 2020 | Sanity pressure in a small team |
| Overcooked | Ghost Town Games, 2016 | Strict, interdependent roles |
| Valheim, Palworld | Iron Gate Studio, 2021; Pocketpair, 2024 | Player-host and dedicated-server models |

## 4. Business Opportunity

The horror multiplayer genre has demonstrated strong commercial viability on PC, with Lethal Company alone selling several million copies as a small indie production, showing that a small team can succeed in this space without AAA-scale budgets.

THE LAST SIGNAL differentiates itself by combining two elements rarely found together in the genre: strict role specialization (as in Overcooked) and light and sanity management under survival-horror pressure (as in Phasmophobia and Lethal Company), set inside an asymmetric 3-versus-1 match (as in Dead by Daylight).

No single survivor can see, repair, and respond to danger at the same time, so communication is structurally necessary, and The Shade has to be defeated with light, not with weapons.

Asymmetric multiplayer is harder to sustain than cooperative play, because the player base is split across queues and balance is difficult. The project treats this honestly: the ranked mode is built on a match engine that also works against an AI-controlled Shade, and balance is tuned with telemetry from real matches.

Running the game as a service adds reasons to return: accounts, ratings, and leaderboards. Cosmetic items and seasonal rewards illustrate possible monetization for a later release. Real-money payments and advertising are outside the scope of this capstone.

As a capstone project, the product demonstrates software engineering across the full stack of an online game: game design and client development, an authoritative multiplayer server on Unity Netcode for GameObjects, a backend API with a database, server orchestration, matchmaking and rating, an admin web portal, and an automated build-and-deploy pipeline, tested with simulated players.

## 5. Software Product Vision

For players who enjoy tense survival-horror experiences, "The Last Signal" is a 4-player online game in which three survivors must reactivate a failing mining vessel's reactor and evacuate before it overloads, while a fourth player, The Shade, hunts them in the dark. Every match, called an Expedition, runs on a dedicated game server allocated by the game's own online platform.

Unlike existing horror games where every player shares an identical toolkit, survivors take one of three specialized roles:

- **Drone Operator:** operates a mobile recon drone equipped with a steerable spotlight and motion radar. The drone is the station's only reliable source of light and threat detection. The Operator must choose where to point it, balancing the needs of teammates in different rooms against the risk of leaving someone in the dark.
- **Lead Engineer:** carries electrical tools and handles the electrical half of each paired repair station: wire-matching and circuit-routing puzzles that must be resolved without triggering a short circuit.
- **Mechanic:** carries mechanical tools and handles the mechanical half of each paired repair station: aligning thermal pipes, regulating steam valves, and stabilizing pressure to prevent an overload.

The fourth player is **The Shade**. It senses survivors by sight and noise, grows stronger in darkness, and is weakened by strong light. Strong light builds up Light Burn, and at a threshold The Shade is stunned and forced to retreat. Survivors win in two coordinated phases: stabilizing the repair stations and activating the ship's Reactor Core, then evacuating to the Airlock before a countdown to meltdown expires. The Shade wins by incapacitating every survivor or by running out the clock.

Behind the game, an online platform lets players sign in, enter a ranked queue, and be placed on a game server automatically. The server decides the outcome of every match and reports it to a backend that records history and updates the ratings of both sides. Administrators use a web portal to manage players and servers and to tune balance values without releasing a new client build. When no human plays The Shade, a finite state machine controls it, which gives the game a practice mode and a fallback.

The figure shows the main components of the platform.

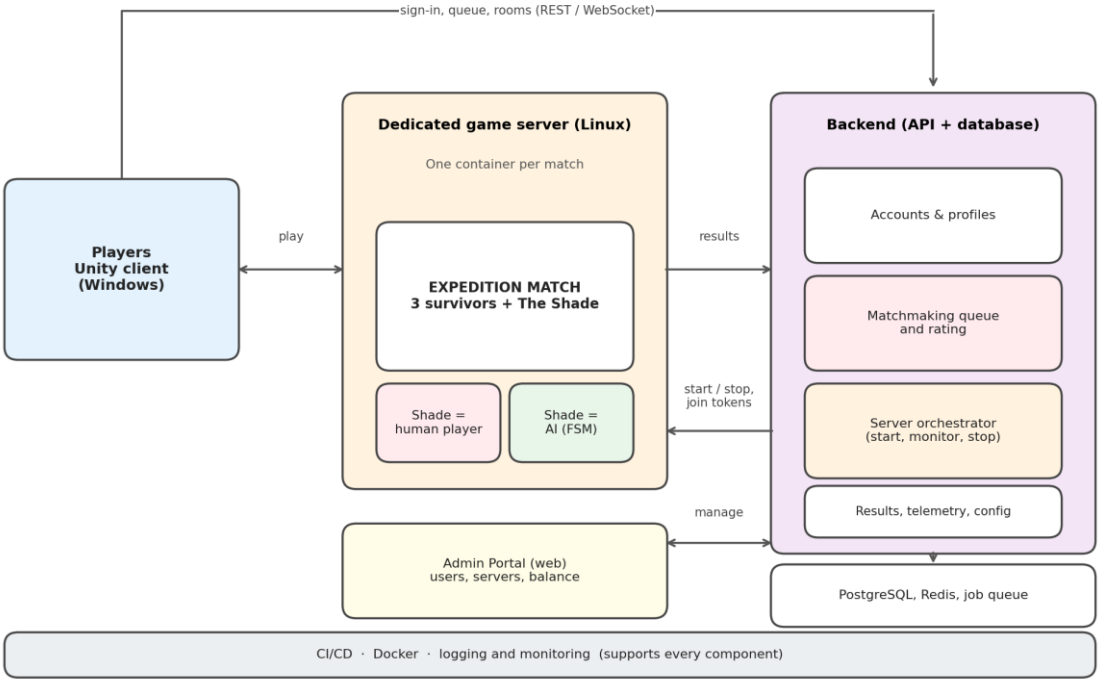



*Figure 2. Platform overview: client, dedicated game server, backend, and admin portal.*

## 6. Project Scope & Limitations

Features are grouped into game features and platform features. Each feature has a tier. **Core** features are required for the capstone MVP. **Extended** features are delivered in the order listed, once the Core works end to end. **Stretch** items are attempted only if the schedule allows.

### 6.1 Major Features

#### 6.1.1 Game Features

| Feature ID | Feature Name | Description / In-Scope Coverage | Tier |
|---|---|---|---|
| FE-01 | Asymmetric 3-versus-1 Matches | Online matches for four players: three survivors against one Shade player. Each match runs on a dedicated server, and the survivors win by evacuating while The Shade wins by stopping them. | Core |
| FE-02 | Survivor Roles | Three survivor roles (Drone Operator, Lead Engineer, Mechanic), each with unique equipment and responsibilities. A fourth survivor role (Logistics) is a stretch goal. | Core |
| FE-03 | Drone Lighting & Awareness | A battery-powered drone, manually piloted by the Drone Operator, providing a directional spotlight and motion radar. | Core |
| FE-04 | Sanity System | Time in darkness lowers survivor sanity, while strong light and nearby teammates restore it. Low sanity makes repairs harder. The Shade gains an advantage in the dark. | Core |
| FE-05 | Paired Repair Minigames | At least five repair stations on the map, each requiring the Engineer and the Mechanic to act simultaneously without triggering failures. A configurable number of stations must be stabilized. | Core |
| FE-06 | Two-Phase Objectives | Stabilize the required repair stations and activate the Reactor Core, then evacuate to the Airlock before a countdown timer expires. | Core |

| Feature ID | Feature Name | Description / In-Scope Coverage | Tier |
|---|---|---|---|
| FE-07 | Playable Shade | A player controls The Shade with movement, two or three abilities, limited vision, and the Light Burn weakness. | Core |
| FE-08 | AI Shade & Practice Mode | An FSM-driven Shade that reacts to light and noise. It powers a singleplayer practice mode (one technician, with the drone following automatically) and replaces a Shade player who disconnects. | Core |
| FE-09 | Space Station Map | A multi-room map with corridors and alternate loop pathways, supporting objective-driven play and tactical retreat. | Core |
| FE-10 | Server Authority & Hidden Information | The server validates every interaction and ability, owns all gameplay-critical state, and sends each player only what that player may see. | Core |

### 6.1.2 Platform Features

| Feature ID | Feature Name | Description / In-Scope Coverage | Tier |
|---|---|---|---|
| FE-11 | Player Accounts & Profiles | Registration and sign-in with token-based sessions, role-based access (Player and Admin), a profile with ratings, and match history. | Core |
| FE-12 | Ranked Matchmaking | A queue with side preference, matches of three survivors and one Shade formed from players with similar ratings, a lobby with role selection and ready check, and a version compatibility check. | Core |
| FE-13 | Game Server Orchestration | The backend starts a containerized dedicated server for each match, issues join tokens, monitors health through heartbeats, frees finished instances, and respects a capacity limit. | Core |
| FE-14 | Authoritative Results & Rating | The server reports each result. The backend validates the match session, applies plausibility checks, and updates each player's Survivor or Shade rating with a Glicko-2 style algorithm. | Core |
| FE-15 | Admin Portal | A web portal to manage users, view running servers and matches, and later edit balance values, review flagged results, and view dashboards. | Core |
| FE-16 | DevOps & Delivery Pipeline | Containerization, CI/CD with automated build and tests, automated deployment, logging, and health monitoring. | Core |
| FE-17 | Leaderboards & Match History | Season leaderboards per side and a history of recent matches. | Extended |
| FE-18 | Custom Rooms | Unranked private rooms joined with a room code. | Extended |
| FE-19 | Telemetry & Balance Dashboard | Match events and dashboards, including the win rate of each side, to tune balance. | Extended |
| FE-20 | Live Balance Configuration | Balance values (ability cooldowns, countdown, sanity rates) are versioned in the backend and edited from the admin portal without a client release. | Extended |
| FE-21 | Player Reports & Moderation | Players can report others; administrators review reports, issue bans, and read an audit log. | Stretch |
| FE-22 | Bot-Filled Matches & Load Testing | Bots reuse the Shade FSM and scripted survivors to fill matches and to load-test the platform. | Stretch |

### 6.2 Limitations & Exclusions

| Limitation ID | Item / Category | Description / Justification |
|---|---|---|
| LI-01 | Single Map Release | The initial capstone release includes a single space station map. Procedurally generated or multiple selectable maps are out of scope. |
| LI-02 | No Real-Money Transactions | Real-money payments, advertising, and third-party store integration are out of scope. Cosmetic items and seasonal rewards are discussed only as a possible later release. |
| LI-03 | No In-Game Voice Chat | In-game voice chat is not implemented; players are expected to use third-party tools during a session. |
| LI-04 | Windows Client, Linux Server | The game client targets Windows PC only, and the dedicated game server runs as a Linux build. Console and mobile platforms are out of scope. |
| LI-05 | No Match Failover | Each match runs on a single game server instance. If the instance fails, the match is aborted without rating change and players return to the lobby. Moving a running match to another server is not supported. |
| LI-06 | Single-Region, Limited Capacity | Game servers run in a single region on a limited number of machines, supporting only a small number of concurrent matches. Automatic scaling across machines is out of scope. |
| LI-07 | Basic Cheat Resistance Only | The server is authoritative over gameplay-critical state and results, and the backend applies plausibility checks. Player movement is still owned by each client, and dedicated anti-cheat software is out of scope. |
| LI-08 | Matchmaking Without a Real Player Population | Matchmaking and rating are verified with simulated players and bots, not with a real player population, so queue behavior at scale is not evaluated. |
| LI-09 | Fixed 3-versus-1 Composition | Ranked matches always use three survivors and one Shade. A four-survivor variant with a Logistics role is a stretch goal. |

### 6.3 Delivery Strategy

The work is staged so that every stage leaves a playable product. Gates decide whether the next stage starts as planned or the scope is reduced.

| Milestone | Condition | If not achieved |
|---|---|---|
| End of week 6 | Dedicated-server spike: a headless Linux build runs, two clients on different networks join, and the current prototype works through the server | Keep the current player-hosted prototype and the backend; reduce the ranked mode to custom rooms |
| End of week 7 | The Shade is playable with one ability and limited vision | Drop the human Shade; keep the AI Shade in a cooperative practice mode |
| End of week 8 | Minimal orchestration runs two matches at the same time | Use a manually started server and reduce the orchestration requirements |
| End of week 10 | Alpha: a full ranked match from queue to result and rating update | Freeze features and prioritize stability for the demonstration |

# II. Project Management Plan

This section describes how the team plans, estimates, executes, and controls the project across the 15-week schedule, given a four-person team with no prior professional game-development or DevOps experience and heavy reliance on AI-assisted development tools.

## 1. Overview

### 1.1 Scope & Estimation

The scope is broken into epics. The first two are complete. Effort for the remaining epics is estimated in person-weeks (one person working full time for one week), based on a four-person team over the remaining ten weeks, which is 40 person-weeks. Complexity reflects both technical difficulty and the team's inexperience with the subsystem. The estimates are rough, with an uncertainty of about 30 to 40 percent, and will be re-calibrated after the week 6 spike.

| # | Epic | Complexity | Estimate (PW) | Planned after cuts (PW) | Status |
|---|---|---|---|---|---|
| 1 | Core Gameplay Prototype (movement, interaction, game loop, prototype map) | Medium | n/a | n/a | Done |
| 2 | Network Infrastructure (player-hosted prototype: Relay, lobby, sync, host authority, disconnects) | High | n/a | n/a | Done; being adapted to dedicated servers |
| 3 | Dedicated Server & Orchestration | High | 6 | 5 | Planned |
| 4 | Backend Platform (accounts, profiles, results) | Medium | 4 | 4 | Planned |
| 5 | Survivor Roles, Drone, Lighting & Sanity | Medium | 5 | 5 | Planned |
| 6 | Paired Repairs, Objectives & HUD | Medium | 4 | 4 | Planned |
| 7 | The Shade (AI bot, playable Shade, hidden information) | High | 6 | 5 | Planned |
| 8 | Matchmaking & Ranked Rating | High | 4 | 4 | Planned |
| 9 | Admin Portal & Moderation | Medium | 2 | 1 | Planned |
| 10 | Telemetry & Balance Dashboard | Low | 1.5 | 1 | Planned |
| 11 | DevOps & CI/CD | Medium | 2 | 2 | Planned |
| 12 | Integration & Alpha | High | 2 | 2 | Planned |
| 13 | Beta Testing, Load Testing & Balance | Medium | 3 | 2.5 | Planned |
| 14 | Release & Capstone Defense | Medium | 3 | 3 | Planned |
| | **Total (remaining epics)** | | **42.5** | **38.5** | Capacity: 40 |

The estimate exceeds the 40 available person-weeks by about 2.5. The planned column applies cuts of about 4 person-weeks (moderation, a minimal bot, minimal orchestration, reduced telemetry, and balance at a "good enough" level), which leaves about 1.5 person-weeks of contingency.

### 1.2 Project Objectives

The project's key objectives for the 15-week schedule are as follows:

1. **Ranked Match End to End:** Deliver a playable ranked 3-versus-1 match on a dedicated server over a real Internet connection, covering the full path from queue to match, result, and rating update.
2. **Server Authority and Hidden Information:** Ensure that clients cannot change gameplay-critical state and never receive information their side may not see, verified by automated and manual tests.

3. **Role Interdependence and Balance:** Demonstrate that survivors depend on each other through paired repairs, and that neither side wins an overwhelming share of matches in beta (target: between 35 and 65 percent for each side).
4. **Platform Operability:** Deploy the backend, admin portal, and game servers through an automated pipeline, with logs and health checks that let the team operate them.
5. **Quality:** Automated tests for rating, sanity, AI determinism, and the API; a load test with simulated players; and no open Blocker or Major defects at submission.
6. **Cost:** Keep hosting within free tiers or student credits.

### 1.3 Project Risks

Key project risks and their mitigations, identified up front given the team's inexperience:

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Converting the player-hosted prototype to a headless dedicated server takes longer than planned (code that assumes the host is a player) | Medium | High | Week 6 spike with a go/no-go gate; review every use of host-specific code; keep client-only code in the view layer |
| Server orchestration (containers, ports, lifecycle) is a new system for the team | High | High | Start with a fixed pool of ports and a minimal orchestrator; fall back to a manually started server for demos |
| Asymmetric balance is hard, and every playtest needs four people | High | High | Use the AI Shade and scripted survivors for solo testing; tune with telemetry; accept "good enough" balance |
| Matchmaking cannot be shown with a real player population | High | Medium | Verify with simulated players and a load test; state the limitation openly (LI-08) |
| Scope exceeds capacity (42.5 estimated against 40 available person-weeks) | High | High | Planned cuts, MoSCoW priorities, weekly review against the milestones |
| Hosting cost or availability problems | Medium | Medium | Student credits and free tiers; run the server on a team machine for development and demonstration |
| Demonstration fails because of network or server problems | Medium | High | Local server fallback on a laptop, recorded video, and a rehearsal before the defense |
| Exposed login and game ports are attacked or abused | Medium | Medium | Join tokens, password hashing, rate limiting, input validation, and firewall rules |

## 2. Management Approach

The team manages the project using Agile Scrum with weekly sprint cycles aligned to the epics, tracked via a Jira board broken into Epics, Stories, and Subtasks.

### 2.1 Project Process

The team follows Agile Scrum with weekly sprints. Each Epic is broken into Stories, each Story into Subtasks, and each Subtask is implemented, manually tested against its own Definition of Done, and merged via pull request before the next Subtask begins. This story-by-story cadence, rather than implementing an entire Epic in one pass, was adopted specifically because the team relies heavily on AI coding assistants (Claude Code) to generate implementation code. Smaller, independently testable increments make it possible to catch AI-introduced errors early rather than after they compound across an entire feature.

### 2.2 Quality Management

Quality is managed through the following practices:

- Mandatory pull-request review before merging to the main branch.
- A manual Play Mode test checklist derived from each Story's Definition of Done, run immediately after any AI-assisted implementation and before merging.
- Automated unit tests for the rating algorithm, Sanity calculation, Light Burn accumulation, and the determinism of the AI Shade.
- Automated API tests for authentication, matchmaking, orchestration, and result handling, run in CI.
- Staged networking tests: a local server first, then a server on a remote machine, then two players on different networks.
- A load test in which simulated players queue, form matches, and finish them, to check matchmaking and orchestration.

### 2.3 Training Plan

No team member had prior Unity or game-development experience at project start. The training approach is just-in-time rather than up-front. The team learned Unity and C# through the Player Controller and Prototype Map stories, then learned each new subsystem immediately before the epic that needs it: Netcode for GameObjects, dedicated server builds, Docker and Linux deployment, and backend and database design. Official documentation and samples are used alongside AI coding assistants for implementation and explanation.

### 2.4 Schedule

| Weeks | Focus | Milestone |
|---|---|---|
| 6 | Decision and spike: dedicated server, backend skeleton, CI/CD | Gate 1: dedicated-server spike |
| 7 | Server conversion, accounts, The Shade prototype | Gate 2: playable Shade |
| 8 | Matchmaking queue, orchestration, drone and sanity | Gate 3: two concurrent matches |
| 9 | Rating, results, paired repairs, objectives | Rating and results work |
| 10 | Integration | Gate 4: alpha, a full ranked match |
| 11 | Admin portal, telemetry, hidden-information hardening | Feature complete for Core |
| 12 | Closed beta | Beta feedback collected |
| 13 | Load testing and bug fixing | No open Blocker defects |
| 14 | Feature freeze, packaging, documentation | Release candidate |
| 15 | Defense preparation and buffer | Rehearsed defense |

## 3. Project Deliverables

Internal deliverables: Software Requirement Specification (SRS), Software Architecture Document (SAD), Game Design Document (GDD), sprint and task board (Jira), versioned source code (GitHub), and the automated test suites.

External deliverables: a Windows 10/11 64-bit game client build, a Linux dedicated server container image, a deployed backend and admin portal with deployment instructions, an installation and operation manual, this Final Project Report, test reports (including the load test), and a demonstration slide deck and trailer for the capstone defense.

## 4. Responsibility Assignments

Rather than assigning responsibility strictly by feature, the team splits work by technical skill area so that no single system stalls the whole team if one member is blocked:

| Area | Primary Responsibility |
|---|---|
| Gameplay | Survivor roles, drone, lighting and Sanity, paired repair minigames, objectives, HUD |
| The Shade and AI | Playable Shade, abilities, hidden information, AI finite state machine and bots, pathfinding |
| Networking and game server | Dedicated server build, join-token validation, state authority, server orchestration, container images |
| Backend and platform | Accounts, matchmaking, rating, results, admin portal, database, CI/CD, deployment, monitoring |
| Shared | Testing, balance, documentation, and preparation of the defense |

All four members pair up on the dedicated-server spike so that networking and deployment knowledge is not held by one person.

## 5. Project Communications

The team communicates via Discord for daily coordination, voice calls, and screen-sharing during pair-programming sessions. Task status and backlog grooming are tracked in Jira (Epics/Stories/Subtasks). Shared documents (SRS, SAD, GDD, meeting notes) are kept in Google Drive/Docs. The supervisor is updated through scheduled weekly check-ins and has read access to the GitHub repository and Jira board.

## 6. Configuration Management

### 6.1 Document Management

Living project documents (SRS, SAD, GDD, and this report) are maintained as version-controlled Markdown or Word files, with major revisions tracked in a revision-history table at the top of each document. The supervisor is given access to the current version at each milestone review.

### 6.2 Source Code Management

Source code is managed in private GitHub repositories using Git, with Git LFS for binary assets (sprites, audio) and a Unity-specific .gitignore and .gitattributes configuration to keep binary noise out of history and enable Unity's smart YAML merge driver. The game and the backend each use a protected main branch (no direct pushes) with short-lived feature branches per Story, merged via reviewed pull requests. Container images are built by the CI pipeline and tagged with the commit and version.

### 6.3 Tools & Infrastructure

The team's development environment consists of:

- **Game engine and IDE:** Unity 6.3 LTS (2D/URP), Visual Studio, Claude Code and Antigravity IDE for AI-assisted development.
- **Multiplayer:** Netcode for GameObjects and Unity Transport, with a Linux Dedicated Server build.
- **Backend (planned):** Node.js with TypeScript, REST and WebSocket, PostgreSQL, Redis, and a job queue.
- **Admin portal (planned):** a React web application.
- **Delivery and operations:** Docker, GitHub Actions for CI/CD, a Linux host for the server and backend, structured logs and health endpoints.
- **Testing:** Unity Test Runner, a JavaScript test framework for the backend, and a load-testing tool with bot clients.

- **Version control and management:** Git, GitHub, Git LFS, UnityYAMLMerge, and Jira.
- **Art and audio:** free or CC0 asset packs (for example Kenney.nl) and Audacity for sound editing.
- **Diagramming:** Draw.io.

# III. Software Requirement Specification

This section specifies the functional and non-functional requirements for THE LAST SIGNAL. Requirements use MoSCoW prioritisation (Must / Should / Could) so scope can degrade gracefully under schedule pressure without abandoning the project's core identity. Must corresponds to the Core tier in Section I.6, Should to Extended, and Could to Stretch.

## 1. Product Overview

THE LAST SIGNAL consists of four cooperating components. The **game client** is a Unity 2D application for Windows. The **dedicated game server** is a headless Linux build that runs one match and is authoritative over it. The **backend** is a web service with a database that handles accounts, matchmaking, ratings, results, and server orchestration. The **admin portal** is a web application for administrators. A singleplayer practice mode runs entirely on the client.

Target environment: Windows 10/11 64-bit for the client, Linux containers for the server and backend, and Unity 6.3 LTS. Online play is designed for Internet latency up to 200 ms. Known constraint: the team has no prior Unity, networking, or DevOps experience, so the architecture favours learnability over sophistication.

The context diagram shows the system and the external parties that interact with it.

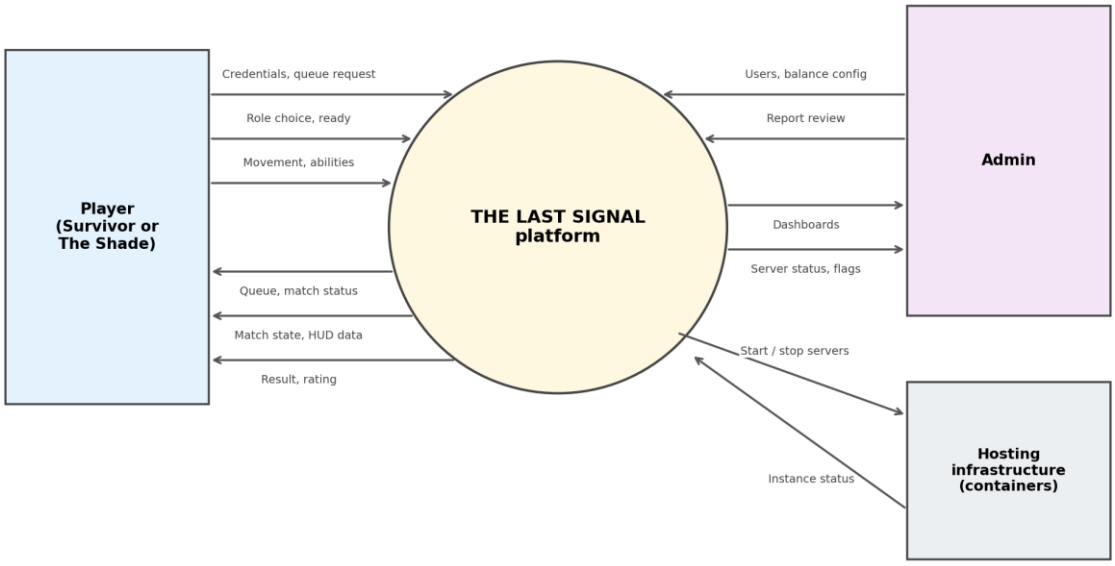



*Figure 3. Context diagram of THE LAST SIGNAL.*

## 2. User Requirements

| Actor | Description |
|---|---|
| Player | A signed-in person who can register, sign in, view a profile and match history, enter the ranked queue, choose a side, ready up in a lobby, leave a match, view the leaderboard, and view match results. |
| Survivor | A Player on the survivor side. Can move and interact, and uses the tools of the chosen role: control the drone (Drone Operator), perform the electrical half of a repair (Lead Engineer), perform the mechanical half of a repair (Mechanic), revive teammates, and use flares. |
| Shade Player | A Player on The Shade's side. Can move, use abilities, and attack survivors. Receives only the information The Shade is allowed to see. |

| Actor | Description |
|---|---|
| Admin | A person who manages the platform: manage users and bans, monitor servers and matches, edit balance configuration, review reports and flagged results, and view dashboards. |
| Hosting infrastructure | An external system that runs the containers for the game servers and the backend. |

The use-case diagram summarizes what each actor can do.

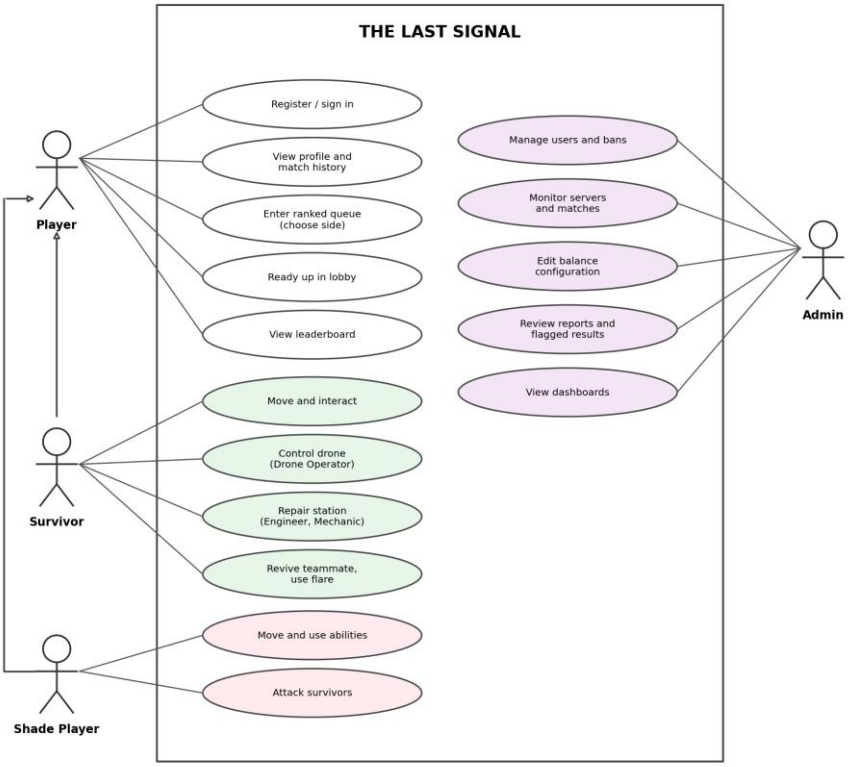



*Figure 4. Use-case diagram of THE LAST SIGNAL.*

## 3. Functional Requirements

Every requirement has success criteria that describe how the team will verify it. Numeric values such as rates, thresholds, and timers are initial tuning values stored in configuration and may change during balancing in the closed beta.

### 3.1 System Functional Overview

Screen flow: Sign-in → Main Menu → Ranked Queue (or Practice Mode) → Lobby (role select, ready) → Match → Result Screen → back to Main Menu. System user roles: Drone Operator, Lead Engineer, Mechanic, The Shade, and the Singleplayer Technician (a merged role for practice mode). Non-screen functions include the matchmaking queue, the server orchestrator, the rating service, the Sanity rules engine, and the AI Shade, which drive what is shown on the HUD and the result screen.

The table below maps each major feature in the Project Introduction (Section 6.1) to the requirements that specify it.

| ID | Name | Specified in |
|---|---|---|
| FE-01 | Asymmetric 3-versus-1 Matches | 3.5 Multiplayer & Server Authority; 3.11 Objectives & Match Flow |
| FE-02, FE-03 | Survivor Roles; Drone Lighting & Awareness | 3.6 Player Roles & Drone |
| FE-04 | Sanity System | 3.12 Sanity, Darkness & Lighting |
| FE-05 | Paired Repair Minigames | 3.10 Paired Repair Stations |

| ID | Name | Specified in |
|---|---|---|
| FE-06 | Two-Phase Objectives | 3.11 Objectives & Match Flow |
| FE-07, FE-08 | Playable Shade; AI Shade & Practice Mode | 3.9 The Shade; 3.18 Practice Mode |
| FE-09 | Space Station Map | 3.8 Space Station Map |
| FE-10 | Server Authority & Hidden Information | 3.5 Multiplayer & Server Authority |
| FE-11 | Player Accounts & Profiles | 3.2 Accounts & Profiles |
| FE-12, FE-18 | Ranked Matchmaking; Custom Rooms | 3.3 Matchmaking & Lobby |
| FE-13 | Game Server Orchestration | 3.4 Game Server Orchestration |
| FE-14, FE-17 | Authoritative Results & Rating; Leaderboards | 3.14 Ranking & Rating; 3.15 Results & Anti-Abuse |
| FE-15, FE-20, FE-21 | Admin Portal; Live Balance Configuration; Moderation | 3.16 Admin Portal; 3.15 Results & Anti-Abuse |
| FE-19 | Telemetry & Balance Dashboard | 3.17 Telemetry & Results Screen |
| FE-16, FE-22 | DevOps; Bots and Load Testing | 4.2 Quality Attributes (NFR-OPS, NFR-TEST) |
| Supporting | Movement and interaction; hazards and health | 3.7 Core Movement & Interaction; 3.13 Hazards & Health |

### 3.2 Accounts & Profiles

Covers registration, sign-in, access control, and the player profile.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-ACC-01 | A player shall be able to register an account with an email address, a display name, and a password. | A new account is created. A duplicate email or display name is rejected with a message. Passwords are stored only as salted hashes. | Must |
| FR-ACC-02 | A registered player shall be able to sign in and receive a time-limited session token. | Valid credentials return a token. Wrong credentials, expired tokens, and tampered tokens are rejected. Repeated failed attempts are rate limited. | Must |
| FR-ACC-03 | The backend shall distinguish Player and Admin accounts and enforce access by role. | Admin functions reject Player tokens, and Player functions work for both roles. | Must |
| FR-ACC-04 | Each player shall have a profile showing display name, Survivor rating, Shade rating, and basic statistics. | The profile loads from the backend and reflects the latest completed ranked match. | Must |
| FR-ACC-05 | A player shall be able to view the history of their recent matches. | The list shows side, outcome, duration, and rating change for each match. | Should |

### 3.3 Matchmaking & Lobby

Covers the ranked queue, match formation, the pre-match lobby, and custom rooms.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-MM-01 | A signed-in player shall be able to enter the ranked queue with a side preference (Survivor, Shade, or Either). | The queue entry is recorded, the player sees a searching status, and the player can leave the queue at any time. | Must |

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-MM-02 | The backend shall form a match of three survivors and one Shade from queued players, preferring players with similar ratings. | A match forms only with the right composition. The allowed rating gap widens as waiting time grows, up to a configured limit. | Must |
| FR-MM-03 | When a match is formed, the system shall notify the four players and move them to the lobby. | All four players see the lobby with the same player list. | Must |
| FR-MM-04 | In the lobby, survivors shall choose distinct roles (Drone Operator, Lead Engineer, Mechanic), and every player shall mark Ready before the match starts. | Duplicate roles are prevented. All players see roles and Ready states in real time. The match starts only when all four are Ready. | Must |
| FR-MM-05 | A game client shall be refused entry to a queue or server when its version differs from the server's. | A client with a different version sees a "Version mismatch" message and cannot join. A client with the same version joins normally. | Must |
| FR-MM-06 | A player who does not become Ready in time shall be removed, and the other players shall return to the queue. | After the timeout the player is removed, and the others re-enter the queue without losing their waiting time. | Should |
| FR-MM-07 | A player who abandons a ranked match shall receive a rating penalty and a temporary queue cooldown. | The penalty and cooldown apply once per abandoned match and are visible on the profile. | Should |
| FR-MM-08 | A player shall be able to create an unranked custom room with a room code, and friends shall be able to join with the code. | The room code is displayed. A player can join by entering it. Custom matches do not change ratings. | Should |

### 3.4 Game Server Orchestration

Covers how the backend starts, secures, monitors, and stops game server instances.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-ORC-01 | When a match is formed, the backend shall start a dedicated game server instance for it in a container. | The instance reports ready before players are told to connect. One instance serves exactly one match. | Must |
| FR-ORC-02 | The backend shall issue each player a single-use, expiring join token for the instance, together with the instance address. | A token works once and only for its own match. Expired, reused, or foreign tokens are rejected by the server. | Must |
| FR-ORC-03 | The backend shall monitor each instance through heartbeats. | An instance that stops sending heartbeats is marked failed within a configured time, and its match is aborted. | Must |
| FR-ORC-04 | The backend shall stop finished or idle instances and release their resources. | After a match ends or all players leave, the instance stops within a configured time and its port can be reused. | Must |
| FR-ORC-05 | The backend shall limit the number of concurrent instances to a configured capacity. | When capacity is reached, new matches wait in the queue with a message instead of failing. | Must |

### 3.5 Multiplayer & Server Authority

Covers how a match runs on the dedicated server, what the server controls, and how disconnects are handled.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-NET-01 | Each match shall run on a dedicated headless server with no local player, and the server shall be authoritative over the match. | The server runs without a display or local player and decides gameplay-critical state. | Must |
| FR-NET-02 | The server shall accept a client only with a valid join token and a matching game version. | Clients without a valid token or with a different version are disconnected immediately. | Must |
| FR-NET-03 | Each player's movement and animation shall be replicated from their own client to the players that may see them. | Remote movement is visible to the right players, and a player's own input has no added delay. | Must |
| FR-NET-04 | Remote players' positions shall be interpolated. | At a simulated latency of 150 ms, remote players move smoothly with no visible teleporting. | Should |
| FR-NET-05 | The server shall be authoritative over doors, repair stations, the Reactor Core, the objective phase, the countdown, hazards, health, Sanity, The Shade, and the match outcome. | All clients display the same values for this state. A modified client cannot change it. | Must |
| FR-NET-06 | Clients shall send interactions and ability use to the server as requests, and the server shall validate each one before applying it. | A request that is out of range, in an invalid state, or against a cooldown is ignored and changes nothing. A valid request is applied and shown identically to all. | Must |
| FR-NET-07 | The server shall send each client only the information that player is allowed to see. | Positions of entities outside a player's visibility (line of sight, light, or radar) are not sent to that client, which is verified by inspecting what a client receives. | Must |
| FR-NET-08 | When a client leaves or loses connection, the match shall continue for the remaining players. | The departed survivor is removed and counted as incapacitated. The remaining players continue without errors or leftover objects. | Must |
| FR-NET-09 | When the Shade player leaves or loses connection, the AI Shade shall take over. | The AI controls The Shade within a short time, and the match is not aborted. | Should |
| FR-NET-10 | When the game server crashes or becomes unreachable, clients shall return to the main menu with a message, and the match shall be recorded as aborted. | No client freezes or crashes. Ratings do not change for the aborted match. | Must |
| FR-NET-11 | A disconnected player shall be able to rejoin the same match within a time window. | A player who reconnects with a valid token returns to their character. | Could |

### 3.6 Player Roles & Drone

Covers the survivor roles and the drone.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-ROLE-01 | The Drone Operator shall pilot a recon drone that provides a steerable directional spotlight and a motion radar. | The Operator aims the spotlight with player input, and the lit area is visible to all players. The radar shows moving entities, including The Shade, within its range on the Operator's screen. | Must |
| FR-ROLE-02 | The drone shall run on a battery that drains while its spotlight and radar are in use and can be recharged with battery cells found on the map. | The battery level is shown on the HUD. At zero battery, the spotlight and radar stop working. Picking up a battery cell restores charge. | Must |
| FR-ROLE-03 | The Lead Engineer shall carry electrical tools and handle the electrical half of each paired repair. | A player with the Engineer toolkit can start the electrical half of a repair, and a player without it cannot. | Must |
| FR-ROLE-04 | The Mechanic shall carry mechanical tools and handle the mechanical half of each paired repair. | A player with the Mechanic toolkit can start the mechanical half of a repair, and a player without it cannot. | Must |
| FR-ROLE-05 | Role tool actions shall use one consistent input scheme across all roles. | The same keys perform interact and use-tool for every role, and on-screen prompts show the key. | Must |
| FR-ROLE-06 | A fourth survivor role (Logistics) shall carry a large backpack and handle scavenging and fire-fighting in an optional four-survivor variant. | In the variant, Logistics carries more items than other roles and can extinguish fires and distribute battery cells. | Could |

### 3.7 Core Movement & Interaction

Covers how players move, collide, interact with objects, and are followed by the camera.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-CORE-01 | Players shall move in eight directions using the keyboard. | Movement works with WASD or the arrow keys. Diagonal movement is not faster than straight movement. Speed is configurable. | Must |
| FR-CORE-02 | The player shall collide with walls, closed doors, and obstacles. | A player cannot pass through solid objects, including at high speed, and does not get stuck on inside corners. | Must |
| FR-CORE-03 | The player character shall face its movement direction and show distinct idle and moving animations. | The facing direction updates as the player moves, and idle and moving states are visibly different. | Must |
| FR-CORE-04 | The system shall detect interactable objects near the player and show an interaction prompt. | The object the player faces (or the closest one in range if none is faced) is highlighted with a prompt. The prompt disappears out of range. | Must |
| FR-CORE-05 | Interactions shall be validated before taking effect. | An interaction only takes effect if the object accepts it in its current state. Other systems, such as the HUD and audio, can react to completed interactions. | Must |
| FR-CORE-06 | The camera shall follow the local player smoothly and stay inside the map. | The camera tracks only the local player's character, moves smoothly, and never shows area beyond the map edge. | Must |

### 3.8 Space Station Map

Covers the single multi-room station map, its doors, and its key locations.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-MAP-01 | The game shall provide one multi-room space station map with rooms connected by corridors and alternate loop pathways. | Every room can be reached from the spawn areas, and at least one loop exists so that players can retreat or flank by more than one route. | Must |
| FR-MAP-02 | The map shall contain doors that players can open and close. | An interact action toggles a door. A closed door blocks movement, and its state is identical for all players. | Must |
| FR-MAP-03 | The map shall define three survivor spawn points, one Shade spawn point, and fixed locations for the repair stations, the Reactor Core console, and the Airlock. | Every player spawns at the right point at match start, and each required objective location exists and is reachable. | Must |
| FR-MAP-04 | The map shall contain both lit areas and dark areas. | Lit areas exist where survivors can recover Sanity, and dark stretches lie between them. | Must |

### 3.9 The Shade

Covers The Shade as a player-controlled role and as an AI-controlled entity.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-SHD-01 | A player shall control The Shade with movement and an attack. | The Shade moves with the same input system as survivors and can attack within range. | Must |
| FR-SHD-02 | The Shade shall have at least two abilities with cooldowns. | Each ability works as configured, and the server enforces the cooldown. Examples are a short dash and a sense pulse that reveals noise sources. | Must |
| FR-SHD-03 | The Shade shall perceive survivors only by line of sight within a field of view and by noise within a noise radius. | Survivors behind walls or outside the noise radius are not shown to the Shade player. Running, failed repairs, and explosions create noise. | Must |
| FR-SHD-04 | Strong light shall weaken The Shade. | Light from the drone spotlight or a flare that hits The Shade builds up Light Burn. At the threshold, The Shade is stunned and retreats for a fixed time (BR-05). | Must |
| FR-SHD-05 | The Shade shall harm survivors it reaches. | A survivor in contact with The Shade's attack loses health (FR-HAZ-01). | Must |
| FR-SHD-06 | The Shade shall gain an advantage in darkness. | For example, The Shade moves faster in dark areas. The bonus is configurable and observable in a test. | Should |
| FR-SHD-07 | An AI-controlled Shade, driven by a finite state machine with Lurk, Stalk/Attack, and Flee states, shall run on the server. | The AI moves between states by fixed rules, detects survivors by field of view and noise, and navigates with 2D pathfinding without getting stuck on walls or corners. | Must |
| FR-SHD-08 | The AI shall separate Stalk and Attack into distinct states, and its behavior shall be deterministic. | Four states exist. The same sequence of sensory inputs always produces the same state transitions, shown by an automated unit test. | Should |
| FR-SHD-09 | Scripted bots shall be able to fill matches and generate load. | Bots can queue, play, and finish a match without a human. | Could |

### 3.10 Paired Repair Stations

Covers the repair stations where the Engineer and the Mechanic work together.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-PUZ-01 | The map shall contain at least five repair stations, and the objective shall require a configurable number of them (initially three) to be stabilized. | Five stations exist. The Reactor Core stays locked until the required number is stabilized. | Must |
| FR-PUZ-02 | Each repair station shall be a paired repair: an electrical half (wire matching or circuit routing) and a mechanical half (pipe alignment, valve and pressure control) performed at the same time by the Engineer and the Mechanic. | Both halves run together. A station is stabilized only when both are completed in the same attempt, and finishing one half alone never stabilizes it. | Must |
| FR-PUZ-03 | A wrong action in a repair shall cause a failure. | A failure resets the attempt, makes noise that The Shade can detect (FR-SHD-03), and can leave a hazard (FR-HAZ-03). | Must |
| FR-PUZ-04 | In practice mode, a single player shall be able to complete both halves of a repair. | One player can stabilize every station alone, and the drone follows and locks onto the repair target. | Must |

### 3.11 Objectives & Match Flow

Covers the two-phase objective, the countdown, and how a match ends.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-OBJ-01 | At match start, all players shall spawn at their spawn points and the objective shall begin at Phase 1. | Four players appear (three survivors and one Shade), and the HUD shows the Phase 1 objective. | Must |
| FR-OBJ-02 | Phase 1: the survivors shall stabilize the required repair stations and activate the Reactor Core. | The Reactor Core cannot be activated before the required stations are stabilized. Activation is validated by the server and shows as active for all players. | Must |
| FR-OBJ-03 | Phase 2: activating the Reactor Core shall start an evacuation countdown. | The countdown starts at activation and is shown on every HUD. The server drives it, so all clients show the same remaining time to within one second. The initial value is configurable. | Must |
| FR-OBJ-04 | The survivors shall win by evacuating to the Airlock before the countdown ends. | When every survivor who is not incapacitated is at the Airlock before the countdown reaches zero, and at least one survivor is not incapacitated, the match ends as a survivor win. | Must |
| FR-OBJ-05 | The Shade shall win if all survivors are incapacitated or the countdown reaches zero. | Either condition ends the match as a Shade win for all players at the same time. | Must |
| FR-OBJ-06 | A match shall also end when a configurable maximum duration is reached, as a Shade win. | Reaching the limit ends the match as a Shade win. | Should |
| FR-OBJ-07 | After a match ends, the result screen shall be shown, and players shall return to the main menu. | The result screen appears for all players, the server instance is released, and no player is left in a frozen game. | Must |

### 3.12 Sanity, Darkness & Lighting

Covers 2D lighting, the Sanity resource of survivors, and the effects of low Sanity.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-SAN-01 | The system shall provide 2D dynamic lighting with shadows that affects what players can see. | Walls cast shadows, and areas outside every light are visibly dark. | Must |
| FR-SAN-02 | The system shall implement at least three ambient-light tiers that affect survivor Sanity: a safe or lit tier (no loss), a dim tier (slow loss), and a dark tier (faster loss). | Each tier changes Sanity at its configured rate. Exact rates are tuning parameters that can be changed without a code change. | Must |
| FR-SAN-03 | The Drone Operator's spotlight shall count as an active light source that can move a survivor into the safe tier regardless of environment lighting. | A survivor inside the spotlight stops losing Sanity. | Must |
| FR-SAN-04 | Sanity shall recover under direct light or near teammates. | Sanity recovers at the configured rate and always stays between 0% and 100%. | Must |
| FR-SAN-05 | Sanity below a defined threshold shall degrade a survivor's success in repair minigames. | Below the threshold, the success window shrinks and the screen shakes. Both stop when Sanity recovers. | Should |
| FR-SAN-06 | Very low Sanity (below 30%) shall add hallucination effects. | At least one of chromatic aberration, audio hallucination, or inverted controls is active below 30% and stops when Sanity recovers. | Could |
| FR-SAN-07 | Failing a minigame at critically low Sanity shall trigger an explosion. | The explosion damages the survivor (initial value 20% of health), ignites the station, and makes a loud noise that The Shade can detect. | Should |

### 3.13 Hazards & Health

Covers player health, fires, and the tools that counter them.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-HAZ-01 | Each player shall have health, and a player whose health reaches zero shall be incapacitated. | Health is shown on the HUD. An incapacitated player cannot move or interact, and all clients show that player as incapacitated. | Must |
| FR-HAZ-02 | Teammates shall be able to revive an incapacitated survivor. | A survivor who interacts with an incapacitated teammate for the configured time returns that teammate to the game with partial health. | Must |
| FR-HAZ-03 | A failed repair shall be able to start a fire at the station. | The fire is visible to all players, damages anyone standing in it, and keeps burning until extinguished. | Should |
| FR-HAZ-04 | Survivors shall be able to extinguish fires. | Interacting with a fire for a short time removes it for all players. | Should |
| FR-HAZ-05 | Each survivor shall carry a limited number of flares that provide temporary bright light. | A used flare lights the area for a limited time, restores Sanity to survivors in it, and counts as a light source against The Shade (FR-SHD-04). | Should |

### 3.14 Ranking & Rating

Covers the skill ratings of players and the leaderboards.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-RNK-01 | Each player shall have two ratings, one for playing Survivor and one for playing The Shade, each starting at a configured initial value. | A new account shows both initial ratings on the profile. | Must |
| FR-RNK-02 | After each valid ranked match, the backend shall update the participants' ratings with a Glicko-2 style algorithm using the match outcome. | Winners' ratings rise and losers' ratings fall, and the change is larger when the result was less expected. Unit tests confirm the values against reference calculations. | Must |
| FR-RNK-03 | Aborted matches and matches ended by a server failure shall not change ratings. | No rating change is recorded for those matches. | Must |
| FR-RNK-04 | The system shall show a leaderboard per side for the current season. | The top players are listed by rating and updated after each valid match. | Should |
| FR-RNK-05 | Ratings shall be softly reset at the start of a new season. | An administrator starts a season. Ratings move toward the initial value, and history is kept. | Could |

### 3.15 Authoritative Results & Anti-Abuse

Covers how results are accepted and protected against abuse.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-RES-01 | The game server shall report each match's result to the backend, and the backend shall accept a result only for a known, unfinished match session, once. | A duplicate or unknown-session result is rejected. A valid result is stored once. | Must |
| FR-RES-02 | The backend shall compute ratings and statistics itself from the reported events and shall accept results only from authenticated servers. | A client cannot submit results. Only an authenticated server can. | Must |
| FR-RES-03 | The backend shall apply plausibility checks to results. | A result with an impossible duration or statistics is flagged for review and does not change ratings. | Should |
| FR-RES-04 | Players shall be able to report another player after a match. | A report is stored with the match and a reason and is visible to administrators. | Could |
| FR-RES-05 | Administrator actions shall be recorded in an audit log. | Each ban, configuration change, and review is logged with the administrator, the time, and the action. | Should |

### 3.16 Admin Portal

Covers the web portal that administrators use to operate the platform.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-ADM-01 | Administrators shall sign in to a web portal that is separate from the game client. | Only Admin accounts can open the portal. Other accounts see an access-denied message. | Must |

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-ADM-02 | Administrators shall be able to search users, view profiles, and ban or unban accounts. | A banned account cannot sign in or queue. | Must |
| FR-ADM-03 | Administrators shall be able to see running servers and matches. | The portal lists each instance with its match, players, age, and recent log lines. | Must |
| FR-ADM-04 | Administrators shall be able to edit versioned balance configuration (for example ability cooldowns, countdown length, Sanity rates). | A new version applies to matches that start afterwards, and the previous version can be restored. | Should |
| FR-ADM-05 | Administrators shall be able to review reports and flagged results. | The portal lists flagged results and reports, and an administrator can mark them as reviewed. | Should |
| FR-ADM-06 | Administrators shall be able to view a dashboard of matches and the win rate of each side. | The dashboard shows matches per day and the survivor and Shade win rates. | Should |

### 3.17 Telemetry & Results Screen

Covers match records and the result screen.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-TEL-01 | The server shall record match events and the outcome (side results, duration, per-player statistics) and send them to the backend. | A record is stored for each valid match. | Must |
| FR-TEL-02 | The record shall include Sanity depletion, repair accuracy, fire incidents, and Shade ability use. | These values appear in the stored match record. | Should |
| FR-TEL-03 | The result screen shall show the outcome, per-player statistics, and each player's rating change. | The result screen shows this information to all four players after the match. | Must |

### 3.18 Practice Mode

Covers the singleplayer mode against the AI Shade.

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| FR-PRC-01 | A single player shall be able to play a practice match as a unified technician against the AI Shade, with the drone following automatically and locking onto the active repair target. | One player can complete the repairs and the objective alone. The drone follows the technician and locks onto a repair target without manual input. | Must |
| FR-PRC-02 | Practice mode shall run locally with no Internet connection and no effect on ratings. | Practice mode works offline and makes no rating changes. | Must |

# 4. Non-Functional Requirements

## 4.1 External Interfaces

| Interface | Description |
|---|---|
| User interfaces | Sign-in and registration screens, main menu, queue and lobby screens (player list, role select, ready toggle), HUD (own Sanity, health, drone battery, objective and countdown, interaction prompt), pause menu, result screen, and a web-based admin portal. |
| Hardware interfaces | Keyboard and mouse on Windows PC. An Internet connection is required for ranked play; practice mode works offline. |
| Software interfaces | Unity 6.3 LTS, Netcode for GameObjects, Unity Transport, a database, Docker, GitHub and GitHub Actions, and Jira. |
| Communication interfaces | The client talks to the backend through a REST API over HTTPS, with WebSocket for queue status. The client talks to its game server over UDP using Netcode for GameObjects. The backend talks to the hosting infrastructure to start and stop containers. |

## 4.2 Quality Attributes

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| NFR-PERF-01 | The client shall run smoothly on standard hardware. | On the team's lowest-spec test machine at 1080p, the average frame rate stays above 50 FPS over a five-minute full match. | Must |
| NFR-PERF-02 | The hosting machine shall run several matches at the same time. | In a load test, at least two concurrent matches run without missed server ticks. This initial target is revised after the first measurements. | Must |
| NFR-NET-01 | Online play shall stay smooth over Internet connections. | Synchronization stays smooth at latency below 100 ms, and the game remains playable at latency up to 200 ms through interpolation. | Must |
| NFR-INT-01 | Gameplay-critical state shall be controlled only by the server. | A client that tries to change door, objective, countdown, or Shade state directly causes no change for other players. | Must |
| NFR-SEC-01 | Accounts and API traffic shall be protected. | The API uses HTTPS, passwords are hashed, session tokens expire, sign-in attempts are rate limited, and no secret is stored in a client build. | Must |
| NFR-SEC-02 | Every backend endpoint shall validate input and check authorization. | Tests confirm that invalid tokens, reused join tokens, duplicate results, and malformed requests are rejected. | Must |
| NFR-REL-01 | The system shall handle disconnects and failures without crashing or soft-locking. | In each of five tests (a survivor leaves, a survivor loses connection, the Shade player leaves, the server crashes, a player quits during the lobby), the remaining players see no crash and no freeze. | Must |
| NFR-COMP-01 | The client shall run on Windows 10 and 11 (64-bit), and the server on Linux. | The release client plays a full match on both Windows versions, and the server image runs on a Linux host. | Must |
| NFR-OPS-01 | The backend, admin portal, and game server shall be packaged as container images and deployed by the pipeline. | A merge to the main branch builds the images, and deployment to the hosting machine needs no manual steps beyond starting the pipeline. | Must |
| NFR-OPS-02 | Each service shall provide structured logs and a health check. | A health endpoint reports each service state, and logs include match and request identifiers. | Should |

| ID | Requirement | Success Criteria | Priority |
|---|---|---|---|
| NFR-DATA-01 | Player data shall be stored in a relational database with versioned schema migrations. | Schema changes are applied by migration scripts kept in version control. | Should |
| NFR-USE-01 | A first-time player shall be able to learn the game without verbal instructions. | In closed beta, at least 80% of first-time testers complete their first repair without help from the team. | Should |
| NFR-MAINT-01 | The code shall keep game rules separate from presentation. | Game-rule classes contain no UI or scene-object code, and the architecture rules are checked during pull-request review. | Should |
| NFR-TEST-01 | Core rules and the API shall have automated tests. | Unit tests for rating, Sanity, Light Burn, and AI determinism, plus API tests, run in CI and pass before a merge. | Should |
| NFR-TEST-02 | Matchmaking and orchestration shall be tested with simulated players. | A load test with at least 20 simulated players forms valid matches and finishes them without orphaned server instances. | Should |
| NFR-COST-01 | Hosting shall stay within free tiers or student credits. | Hosting cost during development and testing stays within the amount the team has agreed. | Should |

## 5. Requirement Appendix

This appendix lists supporting business rules, common requirements, application messages, and the traceability from requirements to the Project Introduction and the Jira board.

### 5.1 Business Rules

| ID | Business Rule |
|---|---|
| BR-01 | A ranked match always has three survivors and one Shade, and it cannot start until all four players have marked Ready. |
| BR-02 | Sanity is always kept between 0% and 100%. |
| BR-03 | Only the server may change gameplay-critical state (doors, repair stations, Reactor Core, objective phase, countdown, hazards, health, Sanity, The Shade, and match outcome). Clients request changes through RPCs. The one exception is each player's own movement and animation, which are controlled by that player's client and replicated to others. |
| BR-04 | Only an authenticated game server may report a match result, and only once per match. Ratings change only for valid, completed ranked matches. |
| BR-05 | Light Burn builds up on The Shade only. When it reaches the threshold, The Shade is stunned and retreats for a fixed 30 seconds, no matter how far the threshold was exceeded. |
| BR-06 | Phase 2 cannot begin until Phase 1 is complete: the Reactor Core can be activated only after the required number of repair stations are stabilized. |
| BR-07 | A match ends exactly once. Once a win or loss is declared, no further outcome can occur in that match. |
| BR-08 | A join token is valid for one player, one match, and one use, and it expires after a short time. |
| BR-09 | The rating gap allowed between matched players increases the longer a player waits in the queue, up to a configured maximum. |
| BR-10 | A player who abandons a ranked match receives a rating penalty and a temporary queue cooldown. |

### 5.2 Common Requirements

- All UI text is in English.

- The game targets Windows PC with keyboard and mouse. Controller, console, and mobile support are out of scope (LI-04).
- Initial tuning values (drain and recovery rates, thresholds, timers, cooldowns, rating parameters) are kept in configuration and may change during balancing.
- Player accounts, ratings, and match records are stored in the backend database. Practice mode stores nothing on the server.
- Out of scope for this release: multiple maps, in-game voice chat, real-money transactions, match failover, and automatic scaling (LI-01 to LI-09).

### 5.3 Application Messages List

| Message | Shown When |
|---|---|
| Signing in… | A sign-in request is in progress. |
| Invalid email or password | Sign-in fails because the credentials are wrong. |
| Searching for a match… | The player is in the ranked queue. |
| Match found | A match is formed and the lobby opens. |
| Starting server… | The game server instance is being started. |
| Version mismatch: please update the game | The client version differs from the server version. |
| Could not connect to the game server | The join token is rejected or the server is unreachable. |
| [Player] disconnected | A player leaves or loses connection during a match. |
| Server connection lost: match aborted | The game server crashes or becomes unreachable. |
| Reactor Core activated: evacuate to the Airlock! | Phase 1 completes and the evacuation countdown starts. |
| SURVIVORS ESCAPED | The survivors win. |
| THE SHADE WINS | The Shade wins. |

### 5.4 Other Requirements

The table below traces each requirement group to the Project Introduction feature it supports and to the Jira work that implements it. The completed work was built for the player-hosted prototype and is being adapted to the dedicated-server architecture. Status reflects the Jira board at the time of writing.

| Requirements | Feature | Jira Epic / Stories | Status |
|---|---|---|---|
| FR-CORE-01 to FR-CORE-06 | FE-01, FE-09 | Epic 1: Story 1.1 (Player Controller), Story 1.2 (Interaction System), camera follow | Done |
| FR-MAP-01 to FR-MAP-03 | FE-09 | Epic 1: Story 1.4 (Prototype Map) | Done (prototype) |
| FR-OBJ-01 to FR-OBJ-05 | FE-06 | Epic 1: Story 1.3 (Basic Game Loop); Epic 03: Story 3.5 | Prototype done; full behavior planned |
| FR-NET-03, FR-NET-05, FR-NET-06, FR-NET-08 | FE-01, FE-10 | Epic 03: Stories 3.4 (Player Synchronization), 3.5 (Host Authority), 3.6 (Disconnect Handling) | Done for the player-hosted prototype; adapting to dedicated server |
| FR-MM-08 (custom rooms) | FE-18 | Epic 03: Stories 3.2 (Lobby), 3.3 (Room Code) | Done for the prototype; to be adapted |
| FR-ORC, FR-NET-01, FR-NET-02, FR-NET-07, FR-NET-10 | FE-10, FE-13 | Epic 3 (new): Dedicated Server & Orchestration | Planned |

| Requirements | Feature | Jira Epic / Stories | Status |
|---|---|---|---|
| FR-ACC, FR-MM, FR-RNK, FR-RES, FR-ADM, FR-TEL | FE-11 to FE-21 | Epics 4, 8, 9, 10 (new) | Planned |
| FR-ROLE, FR-PUZ, FR-SAN, FR-HAZ, FR-MAP-04 | FE-02 to FE-06 | Epics 5 and 6 (new) | Planned |
| FR-SHD, FR-PRC | FE-07, FE-08 | Epic 7 (new): The Shade | Planned |
| NFR (all) | All | Verified during integration, closed beta, and load testing | Planned |